Flux de remboursement, correction et note de crédit

Date : 2026-05-14 **Code :** `src-tauri/src/commands/refund.rs` **Périmètre cert :** M111 (remboursement complet), M112B (remboursement partiel - ticket séparé), UC4118 (correction de paiement), couverture partielle de UC4122 (note de crédit - voir §3).

---

1. Implémentation du remboursement

La commande Tauri `refund_checkout` prend :

- `original_fiscal_event_id` - ligne locale de `fiscal_events` identifiant l'événement N à rembourser.
- `items: Vec<RefundItem>` - quoi rembourser (le ticket complet pour un remboursement total, un sous-ensemble pour un remboursement partiel).
- `employee_insz` - INSZ de l'opérateur, qui doit être le Gérant connecté (`PROFILS_UTILISATEURS_FR.md` §3).

Le flux reflète l'encaissement standard de `src-tauri/src/commands/checkout.rs` :

1. **Chargement de l'original.** Résolution de `fdm_id`, `fdmDateTime`, `eventLabel`, `eventCounter`, `total-Counter` depuis la ligne `fiscal_events` locale. Seuls les événements N sont remboursables ; tout autre label retourne `"Cannot refund event type <X>"`. Sont aussi refusés : un remboursement (un remboursement ne se rembourse pas : on revend), et tout remboursement en mode formation (un événement T ne peut pas consommer la quantité encore remboursable d'une vente réelle).
2. **Contrôles, puis construction de la commande de remboursement.** Chaque article est d'abord contrôlé (présent sur le ticket original, quantité positive, total remboursé sur l'ensemble des remboursements de la vente au plus égal à la quantité vendue), et le support du ticket choisi : un refus ne crée rien. Le compte se fait par produit, prix unitaire et taux de TVA : l'écran envoie la ligne d'origine choisie (`lineId`), et l'article est remboursé au prix de cette ligne. Un produit vendu à plusieurs prix sans ligne désignée est refusé (`refund_line_ambiguous`). Ensuite seulement, création d'une nouvelle `Order` (takeaway) avec des lignes de quantités négatives pour chaque `RefundItem`.
3. **Réservation du prochain numéro de ticket** (peek-then-commit, cf. `STRATEGIE_COMPTEURS_FR.md` §5). Label N.
4. **Construction du `SaleInput`** avec le builder 8-arguments, en passant la `FdmReference` originale comme 8e argument (`Some(&original_ref)`). Le FDM utilise cette référence pour relier le remboursement à sa source - spec POS v2.2 §313 "fdmRef".
5. **Envoi au FDM.** Même appel `FdmClient::sign_sale` qu'un encaissement normal. Le blocage sur déconnexion s'applique à l'identique : sans FDM, le remboursement est refusé et la commande de remboursement est annulée (`voided`).
6. **Archive + clôture atomiques.** Même pattern transaction SQLite unique que checkout : `Fiscal-Counter::next` + `archive_event` + `UPDATE orders SET status='closed'` dans une tx, rollback si l'un échoue. Cf. fix B3 dans `STRATEGIE_COMPTEURS_FR.md` §5.
7. **Impression ticket de remboursement + ouverture tiroir.** Best-effort - non-fatal si imprimante injoignable. Chaque ligne remboursée est imprimée avec sa quantité et son montant négatifs (« -1 Cola -3,04 »), comme la requête M111 les transmet au FDM : le ticket reproduit les données signées.

Si le remboursement s'arrête après la création de sa commande (refus ou indisponibilité du FDM, échec de l'archivage), la commande est annulée (`status = 'voided'`) : elle ne reste jamais ouverte, et ne peut donc pas être complétée puis encaissée comme une vente. Ces règles sont couvertes par `src-tauri/tests/refund_guards.rs`.

M112A vs M112B (remboursement partiel combiné à une nouvelle vente)

Le formulaire de demande de certification exige de déclarer si le POS supporte **M112A** (un remboursement partiel combiné à des items de nouvelle vente sur un même ticket) ou **M112B** (remboursement seul, sur un ticket séparé).

Nous déclarons M112B : la commande de remboursement produit un **événement fiscal séparé** ne contenant que des lignes à quantités négatives, jamais mélangé à des lignes de nouvelle vente. C'est le pattern le plus simple et le plus sûr ; combiner remboursement et nouvelle vente sur un même ticket crée une particularité de piste d'audit que le manuel de certification demande explicitement aux candidats de reconnaître.

Les fixtures M112A et M112B se parsent toutes deux proprement (vérifié dans `uc_fixtures.rs`). Le mode déclaré ne contrôle que le flux UI.

---

2. Correction de paiement (M132, UC4118)

`signPaymentCorrection` est câblé dans `FdmClient::sign_payment_correction` avec la bonne forme `PaymentCorrectionInput`. La fixture se désérialise proprement (M132 dans Basic Set, sous-étapes UC4118 dans Advanced Set). Une commande Tauri dédiée est actuellement absente - les opérateurs doivent utiliser le flux de remboursement à la place. **L'ajout d'une commande de correction de paiement de premier ordre est reporté à une release ultérieure** alongside des autres polissages UI. Le protocole fiscal est satisfait ; le manque est de l'ergonomie opérateur.

---

3. Note de crédit (UC4122)

Selon la spec, une note de crédit est une mutation `signInvoice` avec des montants **négatifs** et un `fdmRef` vers la facture originale. C'est un flux distinct du remboursement - le remboursement utilise `signSale`, la note de crédit utilise `signInvoice`.

Statut actuel :

- `FdmClient::sign_invoice` existe et accepte `InvoiceInput` avec le champ requis `fdm_refs: Vec<FdmReferenceInput>`.
- La fixture UC4122 en quatre sous-étapes se désérialise proprement (vérifié par le test `advanced_set_fixtures`).
- Pas de commande Tauri ni d'UI pour émettre des notes de crédit.

Reporté à la phase Peppol B2B : les factures B2B et notes de crédit partagent le canal Peppol à partir du 2026-01-01, nous construirons la commande note-de-crédit en même temps que la commande facture plutôt que d'étaler le travail.

---

4. Correctifs de bug appliqués

4.1 `restaurant_est_no` vide

`commands/refund.rs:223` - `restaurant_est_no` était hardcodé à `""` sur le `TicketParams` de remboursement, identique au bug de Phase 1 corrigé dans `checkout.rs:205`. Passe désormais `&config.restaurant_est_no`. C'était une vraie régression d'impression - les tickets de remboursement auraient montré un numéro d'établissement vide, ce que le manuel de certification mentionne comme un champ obligatoire du ticket.

4.2 Compteur sans trou (fix B3)

Voir `STRATEGIE_COMPTEURS_FR.md` §5. Le flux de remboursement profite du même refactor peek-then-commit que checkout - test de régression dédié dans `src-tauri/tests/spf_e2e_no_gap.rs::refund_fdm_failure_does_not_`

4.3 `restaurant_est_no` vide dans `copy.rs`

`commands/copy.rs:119` - même bug que ci-dessus, à un troisième endroit (les copies de tickets). Corrigé lors du refactor B1.

---

5. Vérification

```
# Côté fixture : réponses remboursement + note de crédit parsent correctement
cargo test --package pos-fiscal --test uc_fixtures
cargo test --package pos-fiscal --test advanced_set_fixtures

# Côté e2e applicatif : remboursement complet, partiel et fdmRef
cargo test --package pili-pos --test spf_e2e_basic m111_
cargo test --package pili-pos --test spf_e2e_basic m112b_

# Côté no-gap : compteur intact si FDM échoue pendant remboursement
cargo test --package pili-pos --test spf_e2e_no_gap refund_fdm
```

Totaux : totaux par binaire de test dans *RAPPORT_TESTS_BASELINE_FR.md*, régénérés par `scripts/generate_test_rep`